OFFICIAL TECHNICAL STATUS REPORT

Brief! - Social Media Blog API

Application 1.0.2 (revision C) | Report v3 | Verification closed

Project	Brief! - full-stack web application; a message board where every post fits in 255 characters
Author	Ambroise Lumanikio
Application version	1.0.2 (revision C)
Report	OTSR v3 | October 3, 2026 | a delta report: read together with OTSR v2, which holds the technology guide and product directions
Overall status	**Verified.** 67 of 67 automated tests pass on the real stack. Runnable jar builds. One regression found and fixed.

#	Section
1	Summary
2	Automated Test Results
3	Build Warnings Reviewed
4	Regression: Status Dot (found and fixed)
5	Additional Manual Checks
6	Using the App on a Phone
7	Open Items After This Report

© 2026 Ambroise Lumanikio | OneTechly

1. Summary

The one action left open by OTSR v1 and v2 is closed. The full test suite was run on the author's Windows machine against real Javalin 5.6.3, Jetty 11 and H2, on JDK 17: **67 tests, 0 failures, 0 errors, 0 skipped**. The package step then ran the suite a second time with the same result and produced the runnable jar.

Item	v2	v3	Status
Automated suite on the real stack	Not recorded	67 / 67 pass, twice	**PASS**
Runnable jar	Assumed	Built: target\brief-api.jar, BUILD SUCCESS	**PASS**
Server status dot	Believed correct	Was broken by revision B; fixed in revision C	**CLOSED**
Java target	11 (warnings on JDK 17)	17	**DONE**
Phone access	Not documented	Documented and layout verified at 390 px	**DONE**

2. Automated Test Results

Source: the console output of `.\mvnw.cmd test` (19:56) and `.\mvnw.cmd package` (20:31), October 3, 2026, run in the study copy. The study copy compiles to the same classes as the standard copy, so the result holds for both.

Suite	What it proves	Tests	Failures	Time
integration.ContractTest	The eight original requirements plus the new read endpoints, over real HTTP against a real database	30	0	5.1 s
integration.StrictAuthTest	Tokens and ownership: 401 without a token, 403 for someone else's post	8	0	1.1 s
unit.AccountServiceTest	Registration and login rules; hashing; legacy password upgrade	10	0	1.5 s
unit.MessageServiceTest	Post rules, including exactly 255 and 256 characters	8	0	< 0.01 s
unit.PasswordHasherTest	Hash, verify, salt uniqueness	5	0	0.9 s
unit.SessionServiceTest	Token issue, expiry, revoke	6	0	< 0.01 s
Total		**67**	**0**	18.8 s build

What this closes

- **The DAO SQL is correct** on a real database: inserts, generated keys, updates, deletes, the unique constraint.
- **The Javalin wiring is correct:** routes, path parameters, exception mapping to 400, empty bodies, static files (the log shows "Static file handler added").
- **pom.xml is correct:** every dependency resolved, the test database setting was applied, the shade plugin built the jar.
- **The Maven Wrapper works** on Windows.

The two unit suites with no database take about a millisecond each; the suites that hash passwords take about a second. That difference is the cost of 120,000 PBKDF2 iterations, and it is intended.

© 2026 Ambroise Lumanikio | OneTechly

3. Build Warnings Reviewed

None of the warnings indicates a defect. Each was read and classified.

Warning	Where	Meaning	Action
Build path specifies execution environment JavaSE-11 ... no JREs strictly compatible	VS Code Problems	The project asked for Java 11; only JDK 17 is installed. VS Code compiles with 17 in Java 11 mode	Fixed: target is now 17
The compiler compliance specified is 11 but a JRE 17 is used	VS Code Problems	Same cause	Fixed: target is now 17
system modules path not set in conjunction with -source 11	Maven compile	Same cause, reported by javac	Fixed: pom uses maven.compiler.release
Discovered module-info.class. Shading will break its strong encapsulation (17 times)	Maven package	Libraries are merged into one jar, so their module descriptors no longer apply. Normal for a fat jar	None needed
... define N overlapping resources: META-INF/LICENSE, NOTICE, MANIFEST.MF	Maven package	Several libraries ship files with the same name; one copy is kept	None needed
Your Javalin version is 1084 days old	Test log (INFO)	Javalin 5.6.3 is from October 2023	Roadmap: move to Javalin 6

4. Regression: Status Dot (found and fixed)

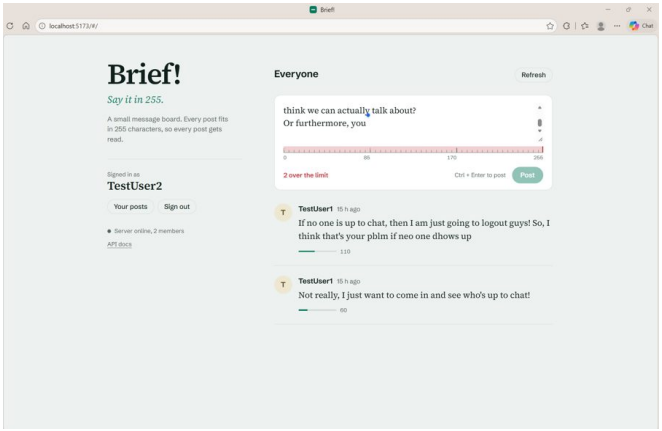
Field	Detail
Symptom	The dot beside "Server online" was grey instead of green. It would also have stayed grey, not red, when the server was unreachable
Found by	Review of the author's screenshots of revision B (the dot is grey in both)
Introduced in	1.0.2 revision B, while adding the "API docs" link
Root cause	The new link's styles were inserted with a find-and-replace on the text ".status-dot {". That text occurs three times in the stylesheet. All three were replaced, which destroyed the selectors of the "offline" and "loading" rules; their bodies became unconditional rules, and the last one (grey) won
Why it was missed	The checks after revision B tested the new page and the user flows, but nothing looked at the dot's colour
Fix	The three rules restored (frontend/src/styles.css)
Verification	In a real browser: computed colour is rgb(10, 122, 94) (green) when online and rgb(192, 42, 51) (red) with API calls blocked
Impact	Cosmetic. The text beside the dot was always correct

> **Lesson recorded**
>
> A bulk find-and-replace must be checked for how many places it matched, not only for whether it matched. More generally: after any stylesheet change, look at every state of the affected element, not only the new one.

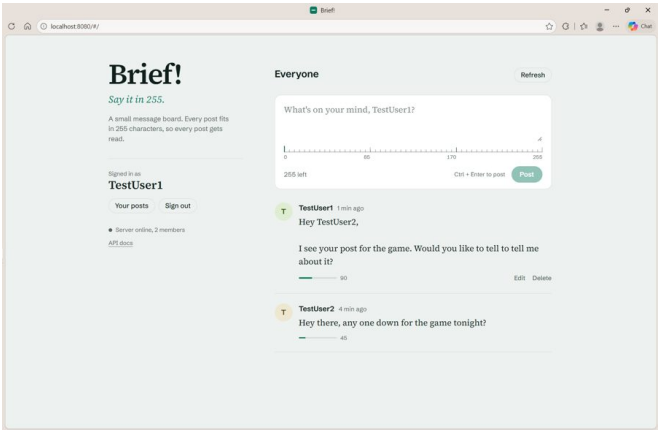
© 2026 Ambroise Lumanikio | OneTechly

5. Additional Manual Checks

#	Scenario	Observed	Result
12	Type past the limit	Ruler fills red, "2 over the limit", Post disabled	PASS
13	Two users in conversation	TestUser1 and TestUser2 posts interleaved, newest first; Edit / Delete only on own posts	PASS
14	Development server (port 5173) against the running API	Signed in and composing through the Vite proxy	PASS
15	Fresh database in a second copy of the project	"Nothing here yet. Sign in and write the first post."	PASS



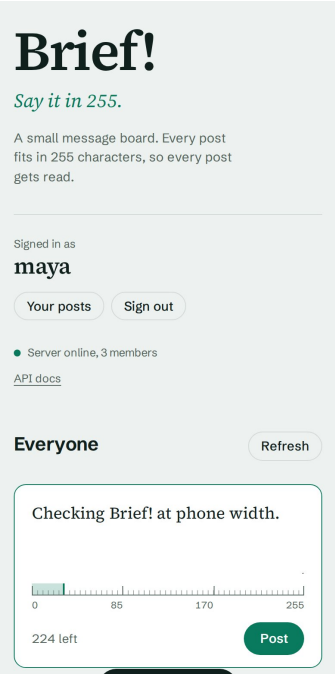
Scenario 12: over the limit. (Revision B: note the grey dot, fixed in revision C.)



Scenario 13: two users.

6. Using the App on a Phone

"localhost" means "this device". Typed on a phone, it points at the phone, where nothing is running. The phone must use the computer's address on the local network.



Brief! at phone width.

Inside your own Wi-Fi

1. Start the backend on the computer.
2. Run `ipconfig` in PowerShell; note the IPv4 Address of the Wi-Fi adapter (for example 192.168.1.23).
3. Connect the phone to the same Wi-Fi network.
4. On the phone open `http://192.168.1.23:8080` (with your own number).
5. When Windows Firewall asks, allow Java on Private networks.

This works because Jetty listens on all network interfaces - the test log shows `0.0.0.0`, not only localhost.

From anywhere

Requires deployment (roadmap 2.1: Docker, a hosted database, HTTPS). Before any public address: HTTPS, a stronger password rule and a login rate limit, all listed as open in OTSR v2.

Layout

Verified at 390 px wide: single column, no horizontal scrolling, composer and ruler usable by touch.

7. Open Items After This Report

Priority	Item	Notes
CLOSED	Automated suite result not recorded (v2, HIGH)	Section 2
MEDIUM	No automated check of the web app's appearance or states	The Section 4 regression would have been caught by one. Candidate: a small Playwright test
MEDIUM	Usernames are case-sensitive; sessions lost on restart; no pagination; hand-maintained openapi.json	Unchanged from v2
MEDIUM	HTTPS, login rate limit, stronger password rule	Unchanged from v2; required before public use
LOW	Javalin 5.6.3 is three years old	Plan the move to Javalin 6, or go straight to the Spring Boot edition (2.0)
LOW	Version 1.0.2 now covers three builds (A, B, C)	The next change should take a new number

With verification closed, 1.0.2 is a sound base. The roadmap in OTSR v2 Section 9 stands.